

Full Concept Brief

AI Study Companion — Hackathon Track: Student Success and Well-being — Sub-focus: Building Effective Study Habits

This document summarizes the full concept as developed so far, including the problem framing, target user, product mechanism, safeguards already considered, and a one-line pitch. It is written to give a reader (human or AI) full context without needing prior conversation history.

1. Problem Statement

First- and second-year Computer Science / Engineering students should ideally use AI as a thinking partner — asking questions to deepen understanding only after attempting a problem themselves. In reality, they tend to ask AI immediately when facing a hard problem, without trying first, and many don't realize how dependent they've become until it is pointed out to them. This erodes independent learning and problem-solving ability over time, and creates an "illusion of understanding" — feeling like they grasp a topic after reading a fluent AI explanation, while being unable to reproduce that knowledge without AI support. No existing tool currently helps students keep using AI to learn while also building gentle, non-judgmental self-awareness of their own usage habits — without rigid bans or shaming.

2. Target User

First- and second-year university students in Computer Science / Engineering programs, taking introductory courses with coding or computational assignments (Intro to Programming, Calculus, Probability & Statistics, Data Structures). This group experiences the problem most sharply: there is a clear gap between "understanding an AI-generated solution" and "being able to solve it independently" — code that runs correctly does not mean the student could write similar code unaided during an exam. This stage is also foundational, since losing independent learning ability in year one has compounding effects on every year that follows.

3. How It Works (Product Mechanism)

3.1 Core Concept

A standalone web app (not a browser extension) where students go directly to ask study questions. Instead of answering immediately like a typical AI chatbot, the app deliberately delays the answer using a Socratic scaffolding flow, then reflects the student's own AI usage patterns back to them weekly. The app calls the Claude API directly and controls the entire conversation flow — this control is essential and is the reason a browser extension (which can only observe third-party AI tools, not control their responses) was ruled out in favor of an independent app.

3.2 Staged Hint Flow (per question)

- **Stage 0 — Student asks a question.** The app does not send it straight to a direct-answer prompt.
- **Stage 1 — Hint generation (API call 1).** A system prompt instructs Claude to act as a Socratic tutor: never solve the problem outright; instead, ask a guiding question or give a minimal directional hint appropriate to the specific problem.

- **Stage 2 — Self-verification pass (API call 2, new safeguard).** Before the hint is shown to the student, it is sent to a second, separate call (can use a cheaper model such as Claude Haiku) with a reviewer-style system prompt: check whether the hint is accurate and leads toward the correct solution; rewrite it if it contains an error or ambiguity. This mitigates the risk of the AI generating an incorrect or misleading hint that the student has no way to independently verify.
- **Stage 3 — Mandatory substantive response (new safeguard).** Before the student can request the next hint, they must type a short response reflecting their own thinking (even a wrong guess, or "I don't know where to start" is acceptable). The app checks — via the model's judgment, not just character count — whether the reply shows genuine engagement. Empty or low-effort replies ("ok", "next") trigger a gentle nudge to share their actual thinking before continuing. This prevents a new, subtler form of dependency: repeatedly clicking "next hint" without ever thinking between hints.
- **Stage 4+ — Escalating hints.** Each subsequent round gives a more specific hint, following the same generate-then-verify pattern.
- **Release stage — Full solution.** Given whenever the student explicitly asks for it (no forced restriction, full autonomy preserved), or after a set number of hint rounds. The full solution is always complete and correct, which also corrects any drift introduced by earlier hints.

3.3 Weekly Reflection Journal

At the end of each week, the app generates a short, gently-worded reflection summary (not a dashboard of red warnings). It tracks:

- **Question type ratio:** classification of each question into Solve (wants the answer directly), Explain (wants a concept explained), or Verify (wants their own work checked) — done via an LLM classification call.
- **Genuine engagement ratio:** proportion of hint rounds where the student wrote real reasoning versus low-effort "next" clicks.
- **Timing / subject patterns:** e.g. direct-answer requests clustering late at night or around a specific subject.

All insights are phrased as open, curious questions (e.g. "Did you notice most of your direct-answer requests happened after 9pm, in Probability?"), never as accusations or judgments. No leaderboards, no comparison with other students, no red color-coding, no pathologizing language (e.g. avoid words like "addiction"). Data is private to the student only — never shared with instructors or the institution.

3.4 Adaptive Scaffolding

Over time, if a student shows they need fewer hints to solve similar problem types independently, the app loosens its scaffolding for that type of problem — shorter, less frequent hints. For unfamiliar problem types, full scaffolding intensity resumes. This mirrors a coach who knows when to step back as a learner gains competence.

3.5 Technical & Cost Notes

- Built as a standalone web app calling the Claude API directly; a lightweight app-side state machine tracks which stage each problem-session is in and decides which system prompt to send on each call — the staged behavior is orchestration logic in the app, not a special model mode.
- Cost optimization: use a cheaper model (e.g. Claude Haiku) for the verification pass and question classification; reserve the stronger model for the main Socratic dialogue generation.

- Hints are designed to be short by nature, which naturally keeps output token cost low compared to a typical AI that writes a full solution immediately.
- Prompt caching can be used for the (fairly long, fixed) Socratic system prompt to reduce repeated input token cost across turns in the same session.

4. Known Limitations / Responsible AI Notes

- **Hint accuracy risk:** an LLM can still generate a subtly incorrect or poorly-directed hint, which is riskier than a wrong direct answer because the student has no easy way to verify it mid-process. Mitigated (not fully eliminated) by the self-verification pass and by always ending with a correct, complete solution.
- **New dependency pattern:** without the mandatory-response safeguard, students could shift from "depending on answers" to "depending on an endless chain of hints" without ever thinking independently. Mitigated by requiring a substantive reply between hints.
- **Students can still bypass the flow** by rephrasing requests cleverly to extract a direct answer early, or by opening a separate AI tool outside the app. By design, the product does not try to hard-block this (consistent with the "no forced restriction" principle) — instead such behavior can become a data point reflected back non-judgmentally in the weekly journal.

5. Mini Pitch (One-Liner)

To help first- and second-year Computer Science / Engineering students use AI as an effective self-learning tool, but who in reality tend to ask AI for answers the moment they hit a hard problem — without attempting it themselves — leading to eroded independent learning and problem-solving skills along with an illusion of understanding they haven't actually earned, we are building **[Product Name]**: an AI study companion that guides students through verified, step-by-step hints before revealing answers, and reflects their own AI usage habits back to them each week — gently and without judgment.